Technical Report

1. Architecture

LiveRecall is a realtime system connecting 3 distinct components over websockets. It consists:

- **Caption source**. This can be a deterministic caption generator (like caption_source.py)
- **Server** (solution_server.py). Each server can connect to exactly one caption source, and N downstream clients.
- **Client**. Each client gets its captions from the server it is connected to.

This report will focus primarily on optimizations made to the server component. We are still using the same message schema as provided in the start kit. The server follows the same high-level architecture as provided in the starter kit, with separate handlers for caption source and client connections. The key changes are detailed below:

1. Fan-out is now decoupled from the source, and each client gets its own outbox

   In the naive implementation, the source handler calls broadcast_naively(), which awaits asyncio.gather() over every client's send() before it reads the next caption. Reading from the source and writing to clients happen in one loop. Now the source handler only appends each caption to every client's ClientOutbox and never awaits client I/O. Each client has its own send_loop task that drains its outbox to its socket. This way, a stalled client blocks only its own task.

2. The server now holds caption state

   In the naive implementation, the server merely forwards captions from the source. Now we maintain a single shared store that every connection reads from. This allows us to reconcile delayed captions, and keep a bounded deque of final segments for clients that need to catch up.

2. Protocol, Revision State and Recovery Semantics

In this section, we'll cover the application protocol used by the server to communicate with the caption source and client, and how it handles revision state and recovery semantics.

Reading caption source

The server exposes a single global ordering function

```
rank(msg) = (segment, final, revision)
```

and a single accepted-state variable LATEST. accept() admits a message from the caption source if and only if its rank is *strictly greater* than that of LATEST. A final for segment *N* dominates every provisional of the same segment regardless of revision number, so a delayed or duplicated provisional can never overwrite a durable final caption.

Recovery and late-join

Finals are additionally added to a FINALS deque trimmed to HISTORY_WINDOW_SECONDS. This is so that a recovering/late-join client can still find its missing finals on reconnect.

On CLIENT_HELLO the server calls missed_finals(hello).

- A *resuming* client (last_segment > 0) receives every final with segment > last_segment.
- A *late-join* client (last_segment == 0) receives finals whose source_ts falls within the requested history_seconds (default 30s, optionally 15 or 60s as specified in the assignment).

It is also worth noting that the HISTORY frame is queued *before* the outbox joins the live broadcast set, so every subsequent final appears in exactly one of the two paths and no final is ever delivered twice.

Client-side deduplication

Our browser client implementation stores captions in a Map<segment, Caption>; an incoming update is applied only when its (final, revision) pair outranks the stored one, so no duplicate lines are ever rendered. The same map feeds the local "What did I miss?" recall panel, which offers 15s / 30s / 60s windows (default 30s) and filters by source_ts ≥ now − W.

3. Backpressure and Overload Policy

Each connected client owns a ClientOutbox with two complementary structures—a FIFO finals deque that is never dropped, and a provisionals dict keyed by segment that coalesces newer revisions over older ones. Pushing a final for segment *N* atomically removes any pending provisional for *N*. The finals deque is capped at MAX_PENDING_FINALS = 256 segments and the dict holds at most one provisional caption per active segment.

Lag measurement

The server infers client-side read backlog from WebSocket pings (we did this because we can't modify the client to support custom capacity messages). Since the websockets library only answers a ping only *after* draining every earlier frame, round-trip time minus the fastest observed RTT gives client read lag:

$$\text{lag} = (\text{now} - \text{ping_at}) - \text{rtt_min}$$

rtt_min is maintained with a two-window sliding minimum (5 s per window) so that an increase in genuine network latency becomes the new baseline rather than being charged forever as client lag.

However, pings cost extra CPU, and we observed in our experiments how pings from an already CPU-bound server will worsen the latency, resulting in a latency cascade. In response, we decided to use a short loop_delay() probe that is subtracted when deciding to evict, preventing a congested server from mistaking its own scheduling delay for a slow client.

Overload path

The high level design is as such:

- By default, finals are prioritized over provisional captions, and provisionals are coalesced.
- When the measured lag is greater than specified threshold (LAG_TARGET_SECONDS), we pause sending provisional captions.
- When the measured lag increases further (beyond EVICT_LAG_SECONDS), or if the number of pending final captions in the queue exceeds MAX_PENDING_FINALS, the server will forcibly close the connection with the client. This forces the client to reconnect and recover the missing captions via a single HISTORY frame instead of waiting to drain from the queue.

Below is a list of the configurable parameters for controlling the server's backpressure and overload policy:

Constant	Value	Description
LAG_TARGET_SECONDS	0.5s	Pause sending provisionals beyond this; keep sending finals only. Coalesced provisional continues to accrue in the dict.
EVICT_LAG_SECONDS	10s	Call transport.abort() to forcibly close the connection when this is exceeded. This forces the client to reconnect and recover the captions through HISTORY.
MAX_PENDING_FINALS	256	Evict for the same reason as above.
BACKOFF_RECOVERY_PER_SECOND	0.1	How quickly the provisional captions sending rate can recover. This is kept low so the websockets write buffer cannot refill with stale data quickly.
HISTORY_WINDOW_SECONDS	90s	Trim FINALS deque when exceeded. satisfies the specified 60s recovery window with additional 30s buffer.
SO_SNDBUF / write_limit	4 KiB / 1 KiB	Kernel + library caps (reduced from the default). This helps us detect congestion quickly so server can start coalescing provisional segments

4. Public Tests and Results

All tests ran on an Ubuntu Multipass VM (2 vCPU, 8 GB) on an Apple Silicon M4 Max. Metrics are as defined in spec §9, with freshness threshold T = 500 ms. All results are median out of 3 runs, as reported by `run_demo.py` provided in the starter kit.

Test	Scenario	P50	P95	P99	Fresh %	Final Fresh %	Final Complete %	Coverage %	Recovery P50
T1	20 clients, 20s baseline	0.8 ms	2.6 ms	3.6 ms	100.0	100.0	100.0	100.0	n/a
T2	100 clients, 40s baseline	2.1 ms	8.9 ms	22.5 ms	100.0	100.0	100.0	100.0	n/a
T3	100 clients, 10% slow @ 500 ms	2.4 ms	32.4 ms	7340 ms	95.4	91.2	100.0	95.0	n/a
T4	100 clients, 20% reconnect @ 15s / 5s	2.2 ms	10.9 ms	18.0 ms	100.0	98.4	100.0	98.3	2.3 ms
T5	100 clients + delay+loss	251 ms	474 ms	763 ms	95.5	95.4	100.0	95.0	n/a

T1 and T2 behave as expected on a healthy local network — all finals arrive, no stale state, and latency is essentially just the localhost round-trip. At 100 clients (T2), backoff never engages and every revision gets through (Cov. = 100%), so the tail is driven by broadcast fan-out rather than any queue pressure. T3's P99 of 7.3 s is the slow cohort sitting at their configured 500 ms sleep per message — the normal 90 clients are unaffected, with P95 staying in the low single-digit milliseconds (see §5.1). Final completeness holds at 100% across both groups because finals skip the backoff gate

entirely. In T4, the 2.3 ms recovery P50 reflects how quickly the HISTORY frame is delivered on reconnect; the slight drop in Coverage to 98.3% is expected — clients miss whatever provisionals were in-flight during their 5 s outage. T5 is the most interesting: P95/P99 jump to 474/763 ms under tc/netem loss, which we attribute to TCP retransmissions interacting with the small write buffer (discussed further in §6). Crucially, final completeness stays at 100% — the finals-first deque ensures durable captions get through even when provisionals are being coalesced away.

5. Discussion on Key Optimizations

This section will cover 2 major optimizations made with discussion on the corresponding ablation results.

5.1 Per-client outbox

The original start kit consists of a single source loop that awaits each ws.send() in turn. This was replaced with a per-client ClientOutbox + send task in the final submission. This protects normal readers from slow readers which would have otherwise hogged up the server in the naive implementation. We observe this in the following ablation test (final submission compared against B1 variant with single outbox):

With 100 clients, 10% deliberately sleep *500/750/1000* ms per message.

slow sleep	Final P95 age	B1 P95 age	Final fresh %	B1 fresh %
500 ms	2.1 ms	1.8 ms	100.0	100.0
750 ms	2.2 ms	1520 ms	100.0	93.0
1000 ms	2.1 ms	6921 ms	100.0	83.3

With 10% of clients sleeping 1000ms, the P95 age rises from 2ms to nearly 7s (~3300× worse) and freshness falls from 100% to 83%. The spike in P95 age suggests that even normal clients (90 percentile) are adversely impacted, since one slow client can block every other client connected to the same server.

5.2 Semantic Congestion Control: Finals-first deque

The second major improvement we made was replacing the single FIFO queue with a queue that prioritizes final caption segments, while merging provisional segments. This helps to prioritize finals (which must be present) over possibly stale provisional segments over a scarce link. In the following ablation test (final submission against B2 variant with FIFO queue), we observe an increase in final completeness % in the implementation that prioritizes final segments.

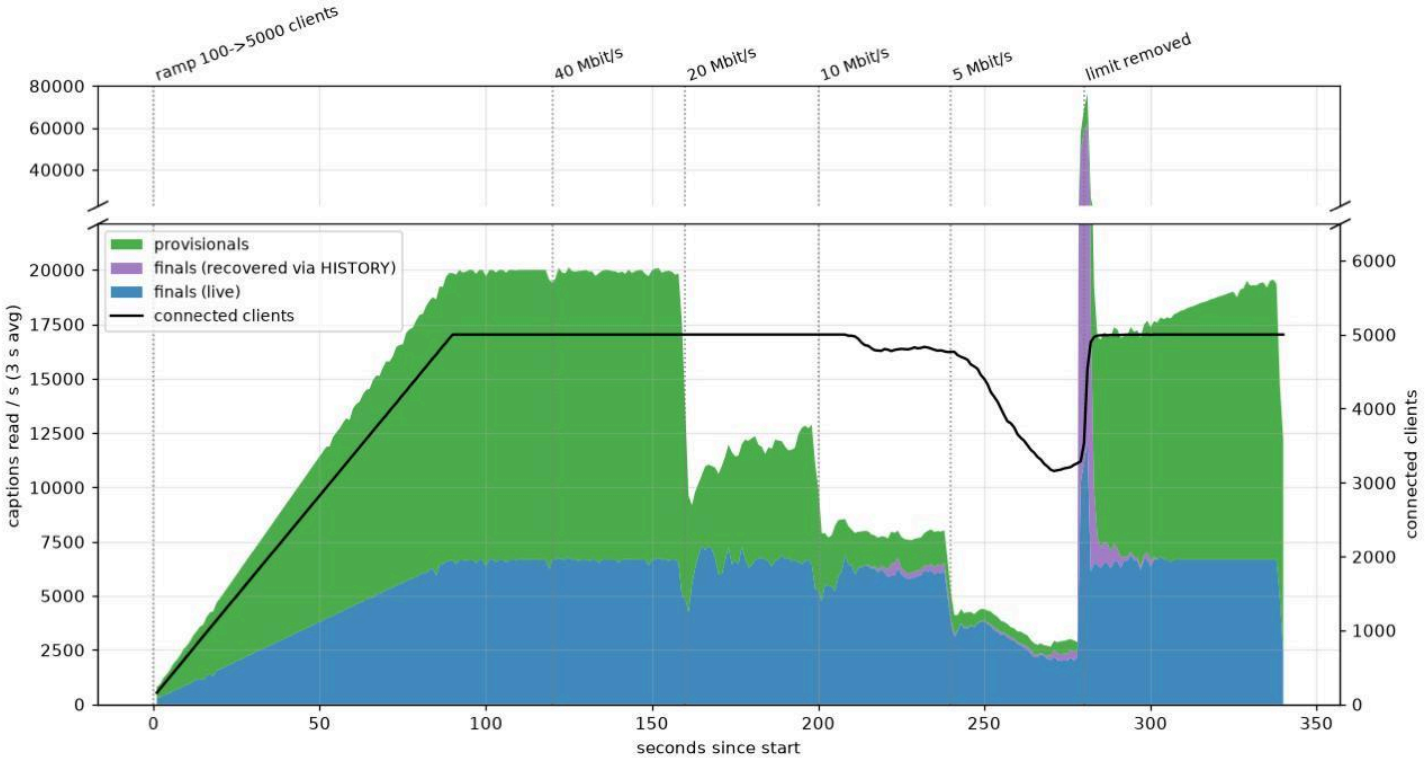
link	Final final completeness %	B2 final completeness %
0.35 Mbit/s	60.3	45.5
0.50 Mbit/s	90.3	65.2
0.75 Mbit/s	98.1	95.8
1.00 Mbit/s	100.0	100.0

At 0.5 Mbit/s our implementation delivers 90% of finals vs B2's 65%, in this case we see more than 25% of durable final transcript lost due FIFO. Below 1 Mbit/s the link is sufficiently constrained and the captions are already stale for both

variants, but *which* captions survive the backlog is a pure policy choice.

5.3 End-to-end bandwidth-stress test

Finally, in this section we combine both mechanisms described in 5.1 and 5.2 and subject the server to a significant bandwidth-stress test. The test harness first ramps up to 5000 connected clients over 90s, then apply loopback tbf caps of 40, 20, 10 and 5 Mbit/s for 40s each, finally removing the cap and giving 60s for recovery.



At 40 Mbit/s the link is unconstrained, and both provisional and final captions are read at their max rate. At 20 Mbit/s we start seeing the effects of the outbox split. The live final segment reads hold out at roughly 6459/s while the server starts shedding provisional captions, whose delivery rate collapses to 5506/s. By 10 Mbit/s, we see a further drop in provisional delivery rate, while live finals are still holding out. At the same time, we observe some clients crossing EVICT_LAG_SECONDS and have their connections forcibly closed by the server, so that they can recover the final segments through HISTORY without waiting on the stale queue (314 finals/s recovered, this is the purple segment). At 5 Mbit/s, about ⅓ of the clients start cycling through evictions and get disconnected, and we see a sustained drop in read rate for both finals and provisional segments.

When the cap is lifted at 290s, every evicted client reconnects within a second and the system returns to 100% fresh-final delivery at full 5000-client load. Many finals are quickly recovered via HISTORY (see the spike in purple graph).

6. Trade-offs & Limitations

Trade-offs

The overload policy explicitly favours freshness over coverage. When backpressure engages, obsolete provisional

revisions are coalesced so that the newest useful state and all finals keep flowing. The 4 KiB kernel send buffer is kept intentionally small for immediate application-layer backpressure signalling (which is what makes coalescing possible before the backlog becomes stale), but that also caps peak per-client throughput on uncongested paths.

Limitations & Future Work

- The extra websocket pings can still be a significant source of latency on a CPU-bound server. An adaptive ping threshold might be helpful to balance between quick lag measurement and CPU-usage. We can also consider replacing the WebSocket ping probe with a read of each socket's TCP_INFO unacknowledged and unsent byte counts; this gives lag estimates without the extra bandwidth cost or the jitter misreading observed on severely constrained links.
- The current hold and eviction threshold is fixed. We could instead derive LAG_TARGET_SECONDS and EVICT_LAG_SECONDS from measured per-client goodput so the server tunes its own freshness ceiling to each link rather than relying on fixed constants
- The implementation assumes a single dispatch server with no horizontal scaling. At some point, we will expect the single asyncio loop in the server to become the bottleneck in this system. We can extend the single-session fan-out into a multi-source lecture-room model with topic sharding and horizontal process-level relay.